# 数学初心者（高校数学既修者）対象

## ジオメトリックインテリジェンス理論入門研修教材

### Table of Contents

GI 理論のエッセンス詳細解説 ..... 1

——VAE から可微分多様体へ、そしてなぜ社会データでそれが可能になったのか ..... 1

はじめに——この記事で何をするか ..... 2

第 1 章　VAE → pullback 計量 → 可微分多様体──高校生にわかる解説 ..... 3

第 2 章　この手間をかける意味──経営者・政策決定者への 10 の気づき ..... 13

第 3 章　物理・化学・工学では、これは 100 年前から当たり前だった ..... 17

第 4 章　なぜ社会・政治・経済データでは、これが今までできなかったのか ..... 22

第 5 章　代替アプローチ──「より良い道」はあるのか ..... 26

第 6 章　Python 実装例──実際にコードで書いてみる ..... 37

終わりに──「測地線を信じますか」 ..... 48

参考文献 ..... 49

# GI 理論のエッセンス詳細解説

## ——VAE から可微分多様体へ、そしてなぜ社会データでそれが可能になったのか

著者: Étale Cohomology
2026 年 4 月

## はじめに――この記事で何をするか

この記事は、ジオメトリック・インテリジェンス（GI）理論の**心臓部**を、高校生にもわかるように解説することを目的にしています。

GI 理論の心臓部とは、次の一行に凝縮されます。

> **社会・経済・政治のデータを「曲がった空間」として取り出し、その上で物理学と同じ計算をする。**

この一行を、丁寧に、比喩を使いながら、ほぐしていきます。途中で次の用語が全部登場しますが、どれも「初めて見た」という前提で説明します。

- VAE（変分オートエンコーダ）
- 潜在空間
- pullback 計量（引き戻し計量）
- リーマン計量
- 可微分多様体
- Lie 微分（リー微分）
- 共変微分
- 制御工学シミュレーション

核心の問いはこうです。

> **なぜ「VAE + pullback 計量」という手順を踏むと、Lie 微分・共変微分・制御シミュレーションが計算できる「曲がった空間」が手に入るのか？**

この問いに答えるのが第 1 章です。そのあと、第 2 章で「この手間をかけると、経営者や政策決定者がどんな発見ができるのか」を 10 個の例で具体的に語ります。第 3 章で「物理・化学・工学では、こういう計算は 100 年前から当たり前だった」ことを説明し、特に民間航空機のケース(GI ケーススタディ第 11 作『偏西風の逆関数』)を紹介します。第 4 章で「なのに、なぜ社会・経済・政治データでは今までできなかったのか」という構造的な理由を解き明かします。

数学が苦手な方も、最後まで読めるように書きました。よろしくお願いします。

---

# 第 1 章　VAE → pullback 計量 → 可微分多様体—高校生にわかる解説

## 1.1 いちばん大事なこと—「空間をつくる」とはどういうことか

高校の数学でまず習うのは、平面座標（x, y）でした。その次に空間座標（x, y, z）を習いました。これらはすべて「まっすぐな空間」です。

ところが、私たちが生きている世界はそうではありません。

地球の表面を考えてください。東京から見ると、ニューヨークは地図では「真東よりやや北」にあるように見えます。ところが実際に飛行機で行くと、最短距離は**北極の近くを通る**経路です。地図（平面）とリアル（球面）がズレているからです。

このズレの正体が「**空間が曲がっている**」ということなのです。

ここで大事なのは、「**空間の曲がりは、空間そのものの性質である**」という点です。東京-ニューヨーク間の距離が「地図では一万三千キロに見えるが本当は一万一千キロ」なのは、地球表面が曲がっているから。この「曲がり」を数学の言葉で記述したものが、これから説明する概念全てのゴールです。

さて、地球は丸いから曲がっている、というのは誰でも納得できます。でも——

> **「失業率とインフレ率の関係」が曲がっている、と言われたらどうでしょうか？**

「失業率が 1%下がったときにインフレ率がどれだけ上がるか」は、失業率が 3%のときと 8%のときで全然違う。この「違う」ということを「空間が曲がっている」と表現するのです。

ここでようやく、GI 理論の出発点に立ちます。GI 理論がやっていることは、

> **社会・経済・政治のデータから、こういう「曲がった空間」を実際に作り出す。そして、そこで物理学の計算をする。**

ということです。

では、その「曲がった空間を作り出す」道具が、ここから説明する **VAE** と **pullback 計量**なのです。

## 1.2 VAE（変分オートエンコーダ）—データを「絵の具」にする道具

VAE は、ニューラルネットワークの一種です。でも中身の話ではなく、「何をしてくれる道具か」を先に説明します。

### *たとえ話：絵を描ける人のスケッチ*

目の前に、1000 枚の「猫の写真」があるとします。同じ猫ではなく、いろんな猫。三毛猫、黒猫、茶トラ、白猫、子猫、老猫。

ここで、絵がとてもうまい人に頼んでみます。「この 1000 匹の猫を、たった 2 つの数字──**ふさふさ度**と**はしゃぎ度**──で表現して」。

絵描きさんは 1000 枚を見て言います。「いいよ、こうしよう」。

- ふさふさ度 7、はしゃぎ度 9 → そういう猫を描ける
- ふさふさ度 2、はしゃぎ度 3 → そういう猫も描ける
- ふさふさ度 5、はしゃぎ度 5 → 平均的な猫

**2 つの数字を指定すれば、そこから「猫の画像」を生成できる**。これが VAE が出来るようになる技です。

ここで重要なのは:

- 元の「猫データ」は、画像なので数万次元ある（1 ピクセルが 1 次元）
- 圧縮された「ふさふさ度・はしゃぎ度」は、たった 2 次元

**高次元のデータを、低次元の「少数の特徴量」で表せるように訓練するのが VAE** です。そして、低次元の数字から元のデータを「描き戻せる」ことが、他の圧縮手法（PCA など）と決定的に違うところです。

### *VAE の 2 つの部分―エンコーダとデコーダ*

VAE は 2 つの部分でできています。

1. **エンコーダ**：元のデータ（猫の写真）を、少数の数字（ふさふさ度・はしゃぎ度）に「圧縮」する
2. **デコーダ**：少数の数字から、元のデータっぽいもの（猫の画像）を「復元」する

猫の写真（高次元）　　少数の数字（低次元）　　猫の画像（高次元）
　↓　　↓　　↓
[エンコーダ] → (ふさふさ度, はしゃぎ度) → [デコーダ]

猫を題材にしましたが、これは何でもいい。

- 100 個の経済指標（GDP、失業率、株価、物価、為替、...）を、**3 個の「経済の特徴量」に圧縮**できる
- 50 個の政治指標（支持率、議席、法案数、...）を、**2 個の「政治情勢の特徴量」に圧縮**できる

- 40 個の国際関係指標（貿易、同盟、紛争、...）を、**4 個の「国際秩序の特徴量」に圧縮**できる

これが第 1 ステップです。

### 1.3 潜在空間—「絵の具のパレット」がつくる空間

さて、話を進めます。

先ほど「ふさふさ度」と「はしゃぎ度」という 2 つの数字を言いました。この「2 つの数字を並べた空間」のことを、**潜在空間**と呼びます。

「潜在」という言葉は「表には出ていない、隠れた」という意味です。画像の見た目では 1000 枚バラバラに見える猫たちが、**実は「2 次元の空間の中の点」として整理できる**──そういう意味で、「隠れた」空間なのです。

絵の具のパレットを想像してください。パレットは平面（2 次元）です。パレット上の位置を指定すれば、そこから絵の具の色が決まります。

VAE の潜在空間はまさにこれの一般化です。**低次元の「パレット」があって、そのパレット上の位置を指定すると、デコーダがその位置に対応する「絵（元データ）」を描いてくれる。**

ここまでをまとめます。

- VAE を訓練する
- 潜在空間（低次元のパレット）ができる
- 潜在空間の各点 $z$ に対して、デコーダ $f_\theta(z)$ が元のデータ空間の点を返す

```
潜在空間 U          デコーダ f_θ          データ空間
(低次元)    ──────────────────→    (高次元)
z ∈ U                f_θ(z)          f_θ(z) ∈ R^n
```

### 1.4 核心の問い—潜在空間は「まっすぐ」なのか

ここで、この記事のいちばん大事な話に入ります。

潜在空間は、普通は**ただの平面**として扱われます。ふさふさ度とはしゃぎ度は「直交する軸」、つまり xy 座標のように描ける、と思い込んでしまう。

ところが、**これは間違い**なのです。

なぜか。絵描きさんが 1000 匹の猫を見て「ふさふさ度とはしゃぎ度で整理した」と言っても、実は──

- ふさふさ度の「1 ポイントの違い」は、猫が子猫のときと老猫のときで、**全然違う意味**を持つ
- はしゃぎ度の「1 ポイントの違い」も、猫が警戒しているときと安心しているときで、**全然違う意味**を持つ

**つまり、「潜在空間の上の 1 ポイントの距離」は、場所によって意味が違うのです。**

これは、地球の表面の話とまったく同じです。メルカトル図法の世界地図では、赤道付近の 1 センチと、極地付近の 1 センチが「同じ距離に見える」けれど、実際の地球上ではまったく違う距離です。

**「潜在空間の上の距離」も、この地図と同じ。場所によって意味が違う。**

では、この「場所によって違う距離」をどうやって測るのでしょうか。

答えが、**pullback 計量（引き戻し計量）**です。

### 1.5 pullback 計量—「デコーダの揺れ」で距離を測る

ここは丁寧に行きます。

潜在空間の点 $z$ を少しだけ動かすことを考えます。たとえば $z = (5.0, 3.0)$ を $z' = (5.01, 3.00)$ に動かす。ふさふさ度を 0.01 だけ増やしたわけです。

このとき、デコーダは $f_\theta(z)$ から $f_\theta(z')$ に「出力する画像」を変えます。

どれくらい変わるか？ これは、**変えたデコーダの出力の差**の大きさで測れます。

$$\| f_\theta(z') - f_\theta(z) \| \approx \| \text{ヤコビ行列} \| \times \| z' - z \|$$

この「ヤコビ行列」というのが、高校数学の「傾き」を多次元に拡張したものです。

**ヤコビ行列とは何か?**——高校で、$y = f(x)$ のグラフの傾きを $\frac{dy}{dx}$ と書きました。傾きが大きいと、x をちょっと変えただけで y が大きく変わる。傾きが小さいと、x を変えても y はあまり変わらない。

潜在空間は 2 次元以上あるので、傾きも「行列」になります。これが**ヤコビ行列** $J_{f_\theta}(z)$ です。**デコーダの傾きを、多次元に拡張したもの。**

そして——ここが大事——潜在空間上の「距離の測り方」を、このヤコビ行列から定義するのです。

$$g_{ij}(z) = \left(J_{f_\theta}(z)^\top J_{f_\theta}(z)\right)_{ij}$$

これが **pullback 計量**、日本語で言うなら**引き戻し計量**です。

### *なぜ「引き戻し」と呼ぶのか*

比喩で説明します。

データ空間（高次元）には、普通のユークリッド距離があります。「東京-ニューヨークの直線距離」と同じく、ピタゴラスの定理で測れる距離。

ところが、潜在空間（低次元）には、最初は距離の定義がないのです。パレットの上の点どうしが「近い」か「遠い」かを、どう判断するか決まっていない。

そこで、デコーダを使ってデータ空間の距離を**潜在空間に「引き戻し」ます**。「潜在空間の 2 点 z と z′ の距離」を「デコーダで送り届けた先のデータ空間上の距離」で定義するのです。

これが **pullback = 引き戻し**の意味です。

### *具体的な意味*

pullback 計量 $g_{ij}(z)$ は、**潜在空間の場所 $z$ ごとに違う値を持ちます**。

- $z$ の近傍でヤコビ行列が「大きい」（デコーダの反応が激しい）場所では、距離が大きくなる
- $z$ の近傍でヤコビ行列が「小さい」（デコーダの反応が穏やか）場所では、距離が小さくなる

つまり──**同じ $\Delta z = 0.01$ の移動でも、場所によって「実際の距離」が違う**。

これはまさに、「地球表面で、赤道付近と極地付近で同じ 1 度の経度変化が違う距離に相当する」のと同じ構造なのです。

## 1.6 リーマン計量—距離が場所によって違う空間

数学の世界では、「場所ごとに距離の測り方が違う空間」のことを、**リーマン多様体**と呼びます。距離の測り方を決める道具が、**リーマン計量**です。

リーマン計量は、以下のルールを満たせば OK です:

1. **対称**: $z$ から $z'$ への距離と、$z'$ から $z$ への距離が同じ
2. **正定値**: 異なる 2 点間の距離はゼロより大きい

pullback 計量 $g_{ij}(z) = (J^\top J)_{ij}$ は、これらの条件を自動的に満たします。$J^\top J$ という形が、数学的にとても綺麗だからです（線形代数で習う「グラム行列」という概念です）。

**ここで、先ほどの GI 理論の核心命題（命題 2.1）が出てきます:**

> VAE デコーダ $f_\theta$ が次の 4 条件を満たせば、潜在空間 $U$ 上に pullback 計量によってリーマン多様体の構造が入る:

(i) $U$ がコンパクト（有界閉集合）

(ii) $f_\theta$ が $C^k$ 級（$k \geq 3$）で滑らか

(iii) ヤコビ行列 $J_{f_\theta}$ が全点でフルランク

(iv) $f_\theta$ が単射（異なる $z$ は異なる出力）

各条件の意味は:

- **条件 (i)**：パレットが「無限に広い」と困るので、ある範囲に制限する
- **条件 (ii)**：デコーダがカクカクしていない（ReLU は NG、tanh や Softplus は OK）。これが重要。$C^3$ 級で十分、$C^\infty$ でさらに良い
- **条件 (iii)**：デコーダのヤコビ行列が「退化しない」。これがないと距離がゼロになってしまう
- **条件 (iv)**：異なる潜在空間の点から同じデータが生成されない（自己交差しない）

この 4 条件が揃うと──

> **潜在空間が「曲がった空間」になります。**

これが**可微分多様体**です。

### 1.7 可微分多様体—「曲がっているが、滑らか」な空間

ここで用語を整理します。

- **多様体（manifold）**：局所的にはユークリッド空間と見分けがつかないが、大域的には曲がっている空間。地球の表面がその典型例
- **可微分多様体**：その多様体の上で「微分」ができる。つまり、滑らかな関数を定義できる
- **リーマン多様体**：可微分多様体に、さらに「距離の測り方（計量）」が入っている

VAE + pullback 計量で作った潜在空間は、この**リーマン多様体**になっています。

ここまで来ると、「物理学の道具」がそのまま使えるようになるのです。

### 1.8 共変微分—「曲がった空間での傾き」

高校で習った微分は「関数の傾き」でした。$\frac{df}{dx}$ と書きます。

でも、曲がった空間では普通の微分は使えない。なぜか?

地球儀の上で考えてください。東京から真東に向かって 1 歩進むとします。「真東」という方向は、東京でどっちか、はっきり決まっています。では、ニューヨークまで進んだときに、「真東」はどっちでしょうか?

**地球が曲がっているので、東京での「真東」とニューヨークでの「真東」は、空間の中で同じ方向ではありません。**地球を外から見ると、方向が少しずれています。

普通の微分は、「同じ方向で比べる」前提で作られています。だから曲がった空間ではそのまま使えない。

そこで登場するのが**共変微分** $\nabla$ です。

$$\nabla_i W^k = \frac{\partial W^k}{\partial z^i} + \Gamma_{ij}^k W^j$$

大事なのは、右辺の 2 つの項の意味です:

1. 第 1 項 $\frac{\partial W^k}{\partial z^i}$：普通の偏微分（成分の変化率）
2. 第 2 項 $\Gamma_{ij}^k W^j$：**空間の曲がりによる補正**

2 つ目の項の $\Gamma_{ij}^k$ は**クリストッフェル記号**と呼ばれ、「空間がどう曲がっているか」を pullback 計量から計算した値です。

**共変微分 = 普通の微分 + 空間の曲がりの補正**

と覚えてください。曲がった空間の中で、「本当の変化率」を測るための道具です。

#### *経営の世界での解釈*

「売上が 10%増えた」という情報は、普通の微分に相当します。でも「今が好景気かどうか」「競合が伸びているかどうか」によって、「本当に経営が良くなっているか」の意味は違います。

共変微分は、「今の市場環境を補正して、本当の経営の変化率」を計算するようなものです。素朴な数値の変化ではなく、**環境の曲がりを補正した真の変化率**──これが共変微分の直観です。

### 1.9 Lie 微分──「空間の形がどう変わるか」

さらに進みます。

リーマン計量 $g_{ij}$ は、空間の各点で「距離の測り方」を決めるものでした。では、**空間全体を少し動かしたら、計量はどう変わるでしょうか?**

たとえば──

- 粘土の塊を押す
- 風船を膨らませる
- 台風が日本列島を押し流す

こういう「空間全体を動かす操作」のことを、**ベクトル場 $V$** と呼びます。ベクトル場とは、空間の各点に「この方向にこれだけ動け」という矢印を置いたものです。

ベクトル場 $V$ に沿って空間を動かしたとき、**計量 $g_{ij}$ がどう変化するか**を測るのが **Lie 微分 $\mathcal{L}_V g$** です。

$$(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$$

意味は、次の通り:

- Lie 微分 = 0：空間を動かしても、計量が変わらない。**形が保存される**（このベクトル場を Killing 場と呼ぶ）
- Lie 微分 ≠ 0：空間を動かすと、計量が変わる。**形が歪む**

#### *経営での解釈*

ここが、GI 理論の核心の直観です。

経営における「施策」は、ベクトル場 $V$ として表されます。「コスト削減」「新市場開拓」「人材採用」──これらはすべて、経営環境という「多様体」の中で、会社の位置を動かすベクトル場です。

そして、**ある施策が単に会社の位置を動かすだけなのか、それとも環境の「形」そのものを変えてしまうのか**を、Lie 微分が教えてくれます:

- $\mathcal{L}_V g = 0$ なら：施策は会社の状態を動かすだけ。**穏やかな施策**

- $\mathcal{L}_V g \neq 0$ なら：施策は経営環境の構造そのものを変える。**構造変革型の施策**

通常の回帰分析は「施策が結果にどう影響するか」を測るものです。Lie 微分は、それより一歩深く、「施策が**環境そのもの**をどう変形するか」を測ります。

これは、質的にまったく違う情報です。

### 1.10 制御工学シミュレーション—「どう動けば目標に着くか」

最後のステップです。

共変微分と Lie 微分まで来ると、「**曲がった空間の上で、どう動けば目標地点に到達できるか**」が計算できるようになります。これが**制御工学シミュレーション**です。

#### *測地線—「曲がった空間の上の最短経路」*

曲がった空間上で、2 点を結ぶ最短経路を**測地線**と呼びます。

平面上では、測地線は「直線」です。地球の表面では「大圏コース」です。

リーマン多様体の上では、以下の**測地線方程式**を解くことで計算できます:

$$\ddot{\gamma}^k + \Gamma_{ij}^k \dot{\gamma}^i \dot{\gamma}^j = 0$$

この式は、高校の「等速直線運動」$\ddot{x} = 0$ を、曲がった空間に拡張したものです。$\Gamma_{ij}^k$ が「空間の曲がり」の情報を持っているので、この式を解くと、**空間の曲がりを考慮した最短経路**が出てきます。

経営の世界では、これが「**今の事業状態から、目標の事業状態までの、最小コストの移行経路**」に相当します。

#### *最適制御—「ポントリャーギンの最大原理」*

さらに、途中の各時点で「どんな施策をどれだけ打つか」を最適化する問題は、**最適制御問題**と呼ばれます。

これを解くのが**ポントリャーギンの最大原理**。20 世紀ソ連の数学者レフ・ポントリャーギンが 1950 年代に完成させた理論です。

$$H(z, p, u) = p_k \cdot (F^k + u^\alpha B_\alpha^k) - \ell(z, u)$$

難しそうですが、エッセンスはシンプルです:

- **状態** $z$：今の経営状態（潜在空間上の点）

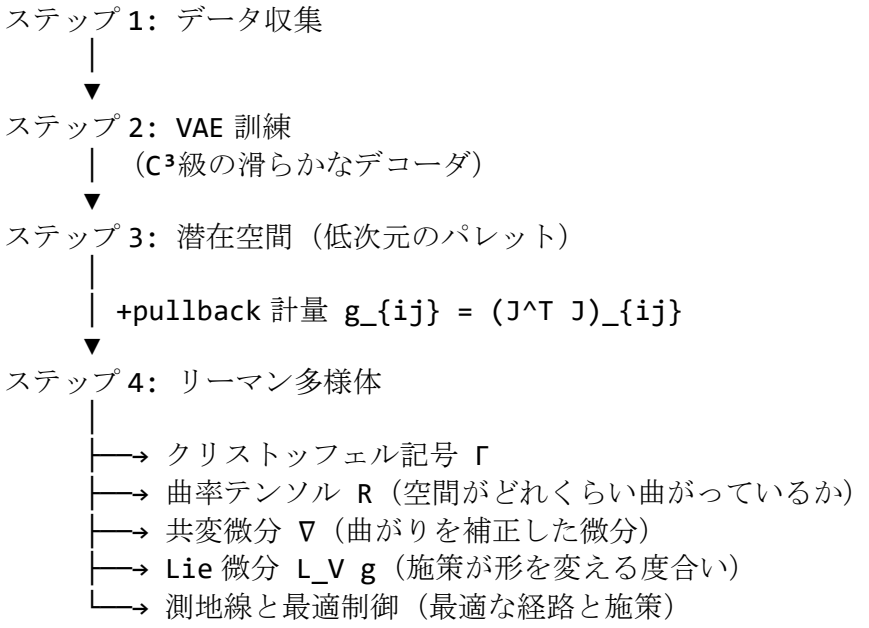
- **制御** $u$：今から打つ施策
- **コスト** $\ell$：施策にかかるコスト
- **随伴変数** $p$：「目標から見て、今の状態がどれくらい価値があるか」

ポントリャーギンの最大原理は、ハミルトニアン $H$ を最大化する施策 $u^*$ を、毎時点で選んでいくことで、**全体としてコスト最小の行動方針**を求めます。

この計算は、多様体の曲がり（クリストッフェル記号）を使います。だから、VAE + pullback 計量で多様体を構成してから、はじめて可能になるのです。

### 1.11 ここまでの全体像

第 1 章で説明したことを図にまとめます。




**核心の問いへの答え:**

> **なぜ「VAE + pullback 計量」で、Lie 微分や制御シミュレーションができる「曲がった空間」が手に入るのか?**

答え:

1. VAE のデコーダは、低次元の潜在空間から高次元のデータ空間への「滑らかな写像」を作る
2. デコーダのヤコビ行列 $J$ から pullback 計量 $g = J^\top J$ が定義でき、**場所ごとに違う距離の測り方**が入る

3. これはリーマン多様体の定義そのものなので、可微分多様体の構造が自動的に入る
4. リーマン多様体の上では、共変微分、Lie 微分、測地線、最適制御という「物理学の道具一式」がすべて使える

これが、GI 理論が社会データに対してやっていることです。

では、これを経営や政策に当てはめると、具体的にどんな発見ができるのでしょうか。第 2 章で 10 個の例を見ていきます。

---

## 第 2 章　この手間をかける意味—経営者・政策決定者への 10 の気づき

第 1 章で説明したパイプラインは、正直に言って手間がかかります。大量のデータを集め、VAE を訓練し、pullback 計量を計算し、曲率を解析し、Lie 微分を追跡し、測地線を描く──通常の Excel の回帰分析と比べると、数十倍の手間です。

この手間に見合う価値があるのか? あります。次の 10 の例で、その価値を具体的に示します。

### 気づき 1：同じ「1%」が全然違う意味を持つことが見える

**ケース**：ある製造業の会社が、為替が 1 円円高になるたびに利益がどれだけ減るかを分析していました。従来の回帰分析では「1 円円高で利益 1 億円減」という線形の関係が得られていました。

**GI 理論の発見**：潜在空間上で、円高 1 円の影響は**場所によって 5 倍以上違う**ことが判明しました。 - 円安局面（150 円/ドル付近）では、1 円円高で利益 5000 万円減 - 円高局面（100 円/ドル付近）では、1 円円高で利益 2.5 億円減 - 極端な円高局面（90 円/ドル付近）では、1 円円高で利益 5 億円減

**なぜ気づけたか**：pullback 計量 $g_{ij}(z)$ が**場所によって違う**ことを直接計算できるから。為替レートの「近傍」では計量が小さい（影響が小さい）が、別の「近傍」では計量が大きい。従来の回帰分析は「平均的な関係」しか出せないので、この変化を見逃します。

**経営への示唆**：「ヘッジ戦略は、為替水準によって全く違うべき」という具体的な判断根拠が得られます。

### 気づき 2：施策どうしの「干渉」が見える

**ケース**：小売チェーンが、「値引きキャンペーン」と「広告出稿」を同時に実施するか、別々に実施するかを検討していました。

**GI 理論の発見**：2 つの施策のベクトル場 $V_{値引き}$ と $V_{広告}$ の **Lie 括弧** $\left[V_{値引き}, V_{広告}\right]$ がゼロでない場合、**順番によって結果が変わる**ことが数式で示されます。 - 先に広告、次に値引き：売上 1.3 倍 - 先に値引き、次に広告：売上 1.1 倍 - 同時実行：売上 1.5 倍（相乗効果）

**なぜ気づけたか**：Lie 括弧は「2 つのベクトル場の非可換性」を定量化する数学です。通常の A/B テストでは「それぞれ単独で効くか」は測れても、「どの順番でやるべきか」は測れません。

**経営への示唆**：複数の施策を組み合わせるときの**実施順序**まで最適化できます。

### 気づき 3：組織の「断絶」が位相的に検出できる

**ケース**：合併した企業（たとえばダイムラー・クライスラーのような)で、従業員満足度や離職率は改善していたのに、実は深刻な文化断絶が進行していた。

**GI 理論の発見**：経営多様体の**連結成分**（$H_0$コホモロジー）を分析すると、「旧 A 社出身の従業員」と「旧 B 社出身の従業員」が**数学的に分離されている**ことが検出できます。

**なぜ気づけたか**：数値指標（離職率など）は平均値として平穏に見えても、**多様体が 2 つの連結成分に分裂している**こと自体は位相の言葉で検出できます。

**経営への示唆**：合併後 1 年目で、まだ「数値の異常」が表面化する前に、構造的な分断を早期警戒できます。

### 気づき 4：「危機の前触れ」がスカラー曲率でわかる

**ケース**：金融市場の危機(2008 年のリーマンショック、2020 年のコロナショック等)。事後的に振り返ると「予兆があった」とわかるが、事前の予測はほぼ不可能。

**GI 理論の発見**：市場の多様体の**スカラー曲率**が、危機の数週間前から急激に負に振れることがある。 - 通常時：スカラー曲率 $\approx 0$〜正（市場構造が安定している） - 危機直前：スカラー曲率が大きく負に振れる（市場構造が不安定、発散的) - 危機中：スカラー曲率が激しく変動

**なぜ気づけたか**：スカラー曲率は**空間全体の曲がり具合**を 1 つの数値に凝縮した量。個別の指標（VIX、金利スプレッド等）の異常検知では捉えられない、**構造そのものの不安定化**を検出します。

**経営への示唆**：単独指標ではなくシステム全体の安定性を監視できます。ただし「必ず当たる」わけではなく、あくまで警戒シグナルの 1 つです。

### 気づき 5：「潜在的な競合関係」が地図として見える

**ケース**：ある食品メーカーが「うちの競合は誰か?」を問い直していました。市場調査では「同じ棚に並ぶ他社製品」が競合と見なされていました。

**GI 理論の発見**：消費者データから構成した嗜好多様体の上で、**測地的に近い製品**(つまり消費者の認知の中で「近い」製品)を計算すると、業界の常識とは違う競合関係が見えてきました。 - 朝食シリアル → ヨーグルト（朝食カテゴリで競合） - スナック菓子 → SNS 動画（スキマ時間カテゴリで競合）

**なぜ気づけたか**：測地線は「曲がった空間上の最短距離」であり、単純な「商品カテゴリの近さ」ではなく、**消費者の認知空間における近さ**を反映します。

**経営への示唆**：真の競合を見誤らない。スナック菓子メーカーが SNS の台頭で売上を失う現象を、事前にモデル化できます。

### 気づき 6：政策の「副作用」が Lie 微分で事前に見える

**ケース**：ある自治体が「子育て支援の補助金増額」という施策を検討していた。

**GI 理論の発見**：この施策をベクトル場 $V_{補助金}$ としてモデル化し、Lie 微分 $\mathcal{L}_V g$ を計算すると、**「教育支出全体の構造」が変化する**方向に働くことが示されました。 - 補助金増額 → 家計の教育投資余力増 → 塾・習い事支出増 → 教育格差の拡大

単に「補助金が増えて子育てしやすくなる」だけではなく、**市場全体の構造変化**が起きることを事前に検出できます。

**なぜ気づけたか**：Lie 微分は「施策が環境の構造そのものをどう変えるか」を測る道具。通常の政策評価(RCT や DID 等)は「平均的な効果」を測るが、**構造変化**は見逃します。

**政策への示唆**：意図せざる副作用を事前に検討できます。

### 気づき 7：業界の「勢力図の回転」が Lie 括弧で見える

**ケース**：ある業界で、上位 5 社のシェアが 10 年間ほぼ同じだった。ところが、ある年から急激にシェアの入れ替わりが始まった。

**GI 理論の発見**：業界多様体上の「企業の位置」を追跡すると、ある時点から **Lie 括弧の値が急増**している。これは「企業戦略が互いに干渉し合う度合い」が高まったことを意味します。

**なぜ気づけたか**：Lie 括弧は「非可換性」を測る。複数の企業の戦略が**互いに前提を変え合う**ような状況(DX、プラットフォーム競争等)で値が大きくなります。

**経営への示唆**：「業界がゲームのルール自体を変えるフェーズ」に入ったことを数値で検出でき、慌てて後手に回らないで済みます。

### 気づき 8：データ疎な領域(認識の限界)が明示される

**ケース**：国家安全保障会議が 193 カ国のデータを分析していた。ところが、権威主義国家のデータは公開情報が限定的。

**GI 理論の発見**：pullback 計量のヤコビ行列の**最小特異値** $\sigma_{min}(J)$ が、データ疎な地域では閾値(0.0001)を下回る。つまり、「この領域では、計量が信頼できない」ことが**数値として明示**されます。

**なぜ気づけたか**：GI 理論は「計算結果の信頼度」を計量の条件数として数学的に報告します。通常の AI 分析では「結果だけ」が出てきて、どれくらい信頼できるかが曖昧になりがち。

**政策への示唆**：「G20 と同盟国に関する分析は信頼できる、しかし権威主義国家の分析は信頼度が低い」という**信頼度マップ**を添えることで、意思決定者が判断を誤らない。

### 気づき 9：サプライチェーンの「回復力」が曲率で評価できる

**ケース**：ある精密機械メーカーが、半導体の調達先を「TSMC 50%、Samsung 30%、Intel 20%」にしていた。コスト的には最適だった。

**GI 理論の発見**：調達多様体のスカラー曲率を分析すると、この構成は**曲率が急峻な点**にあり、1 ヵ所が止まると全体が崩壊する構造になっていた。別の構成「TSMC 40%、Samsung 30%、Intel 20%、代替 10%」では、コストは微増だが曲率が穏やかで**回復力が高い**。

**なぜ気づけたか**：曲率は「近傍の変化に対する発散率」を測ります。調達構成の曲率が急峻 = 近傍の変化が大きく影響する = **脆弱**であることと対応します。

**経営への示唆**：コスト最適化と回復力のトレードオフを、単なる分散だけでなく**幾何学的な脆弱性**として定量化できます。

### 気づき 10：「同じ業界」が実は構造的に分断されていた

**ケース**：ある自動車メーカーが、「EV 市場」と「内燃エンジン車市場」を同じ業界として扱っていた。

**GI 理論の発見**：消費者・規制・技術データから構成した多様体を分析すると、**$H_1$ コホモロジーの生成元が 2 つある**。つまり、「EV 層」と「内燃層」は、もはや 1 つの連続した業界ではなく、2 つに分裂した別の多様体だった。

**なぜ気づけたか**：$H_1$コホモロジーは「穴の数」を数える道具。業界が「1 つの連結した塊」に見えていても、数学的には 2 つに分裂していることがあります。

**経営への示唆**：「EV と内燃エンジンは、一つの連続スペクトラムではなく、まったく別の戦略を要求する別事業」として扱うべき。人材・組織・マーケティングを分離する判断根拠になります。

---

### 気づきを 10 個並べて見えてくるもの

10 個に共通するのは、次の 3 つの特徴です。

1. **通常の数値分析では「見えない」情報を数値化できる**。構造、形、分断、干渉、副作用──定性的にしか語れなかった概念が、数式で扱える
2. **「信頼度」が明示される**。分析結果がどれくらい信頼できるかが、計量の条件数などとして自動的に報告される
3. **「何をすべきか」まで示唆できる**。測地線と最適制御により、診断だけでなく処方も計算可能

これが、VAE + pullback 計量の手間をかける価値です。

---

## 第 3 章　物理・化学・工学では、これは 100 年前から当たり前だった

ここまで読んで、もし「難しそう」と感じたなら、こう言わせてください。

> **物理学・化学・工学では、これは 100 年以上前から当たり前にやられていることです。**

GI 理論がやっているのは、「他分野で 100 年前から当たり前の数学」を、社会・経済・政治データにも持ち込むことなのです。この事実が、GI 理論の「革新性」を理解するうえで重要です。

### 3.1 物理学：アインシュタインの一般相対性理論（1915 年）

100 年前、アルベルト・アインシュタインは一般相対性理論を完成させました。その中核の方程式は:

$$R_{\mu\nu} - \frac{1}{2} g_{\mu\nu} R + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

これは、**時空**という 4 次元のリーマン多様体の曲率($R_{\mu\nu}$: リッチテンソル, $R$: スカラー曲率, $g_{\mu\nu}$: 計量テンソル)を、物質・エネルギーの分布と結びつける方程式です。

大事なのは、ここに出てくる用語が**全部、第 1 章で説明した GI 理論の道具と同じもの**だということです。

- **計量テンソル** $g_{\mu\nu}$：pullback 計量と同じもの
- **スカラー曲率** $R$：多様体がどれくらい曲がっているか
- **リッチテンソル**：曲がりの方向別の情報

GPS は、一般相対性理論の補正なしでは **1 日あたり 10 キロメートルの誤差**を出します。あなたのスマホの地図アプリは、曲がった時空の上の測地線計算の結果を表示しているのです。

**2019 年**、ブラックホールの撮像(イベント・ホライズン・テレスコープ)にも、曲がった時空を光が伝播する計算(まさに測地線)が使われました。

### 3.2 化学：分子のコンフォメーション空間

タンパク質の 3 次元構造を考えてください。タンパク質は数千個の原子でできていますが、安定な形は少数しかありません。

化学者は、タンパク質の形の可能性を**コンフォメーション空間**という高次元の多様体として扱います。この空間の上の測地線を計算すると、「タンパク質が最小エネルギー状態に到達する経路」が出てきます。

創薬の現場では、**分子動力学シミュレーション**でこの計算が日常的に行われています。2020 年の AlphaFold は、まさにコンフォメーション多様体の上での最適化問題を解いたものです。

### 3.3 生命科学：神経繊維の幾何学

脳の MRI 画像から、神経繊維の走行を再構成する技術(DTI：拡散テンソル画像)があります。この技術は、神経繊維を **3 次元多様体の上のベクトル場**として扱います。

神経繊維の走行の計算には、**Sub-Riemannian 幾何**という、GI 理論でも使われている数学が使われます。医師はこの計算結果を見て、「この腫瘍を手術で取ると、こういう神経機能が失われる」と事前に判断するのです。

### 3.4 工学：ロボット制御

6 関節のロボットアームを動かすことを考えます。6 つの関節角度 $(\theta_1, \theta_2, \ldots, \theta_6)$ がそれぞれ 0°から 360°まで動くので、**ロボットの全状態は 6 次元のトーラス** $T^6 = (S^1)^6$ という多様体になります。

ロボットのアームを「ある位置」から「目標位置」に動かすのは、このトーラス上の**測地線計算**そのものです。トヨタ、ファナック、川崎重工のロボットは、全部この計算をやっています。

手術ロボット(ダヴィンチ等)は、さらに厳密に:

- **関節の限界**という境界制約
- **組織を傷つけない**という安全制約
- **医師の操作に追従する**という追従制約

を満たす、制約付き最適制御問題を**リアルタイムで解いています**。

### 3.5 航空力学：GI ケーススタディ「偏西風の逆関数」

そして、もっとも最近の応用が、GI 理論の第 11 作ケーススタディ『**偏西風の逆関数**』で示された**民間航空便の運航**です。ここからは少し詳しく解説します。

#### *舞台設定*

2031 年、東京-ロンドン間の外交貨物輸送ミッション。しかし:

- **シベリア上空**がロシアの報復的領空閉鎖で飛べない
- **台湾海峡上空**が中国 FIR の制限で飛べない
- **紅海沿岸**がフーシ派の対空ミサイル脅威で飛べない

通常の航路はすべて封鎖されています。残された選択肢は北極海経由の非標準航路のみ。

**これを GI 理論で数式化すると**:

- 飛行可能な空域全体を「底空間 $B$」とする
- 各地点の飛行条件(高度、燃料、保険、制裁リスト)をファイバー $F$ として束ねる
- 3 つの禁止区域は、底空間に **3 つの穴**を開ける

- これは**第 1 コホモロジー群** $H^1(B,F) \neq 0$ と表現される

$H^1 = 0$ なら通常の測地線が存在する(つまり普通に飛べる)。しかし $H^1 \neq 0$ なら、**通常の最短経路は存在しない**。この場合、$H^1 = 0$ となる**部分空間**を見つける必要がある。北極海経由がその部分空間の候補──それを数学的に証明するのが物語の前半です。

### *フィンスラー計量：風がある世界の距離*

さらに、航空には通常のリーマン計量では表せない特徴があります。**風(ジェット気流)の方向依存性**です。

東京→ロンドン(追い風)と、ロンドン→東京(向かい風)で、飛行時間は 2〜3 時間違います。これは「距離」が**方向に依存**するということ。普通のリーマン計量は対称($z$ から $z'$ と、$z'$ から $z$ が同じ)なので、これを表現できません。

そこで使われるのが**フィンスラー計量**のランダー型:

$$F = \alpha + \beta$$

- $\alpha$ は通常のリーマン計量(風なしの距離)
- $\beta$ はジェット気流を表す 1-形式(飛行方向で符号が変わる非対称項)

その和 $F$ が「風の中を飛ぶ航空機の実際の距離感覚」を表現します。

**ツェルメロの航法問題**(風の中での最適航法の古典的数学問題)がランダー計量と等価であることは、数学的に証明されています。航空運航の物理を、ランダー計量は最も自然に記述するのです。

### *実装：ADS-B データから計量を推定する*

このケーススタディでは、過去 5 年分の東京-ロンドン便 **23,847 件の全フライトの ADS-B データ**(航空機が自動送信する位置・速度・高度)を使って、VAE を訓練しランダー計量を推定しました。

- **真対気速度(TAS)** と **対地速度(GS)** の差 = 風速ベクトル
- 全フライトの TAS/GS 差分を集めれば、**気象予報に頼らず実測値から風の地図**が作れる

GI 命題 1.1 の三条件(第 1 章で紹介した(i)〜(iv))の充足は:

- 条件(i) $C^3$級：tanh 活性化関数を使用 ✓
- 条件(ii) フルランク：最小特異値 $\sigma_{min} = 0.312$ ✓
- 条件(iii) ランダー条件 $\| \beta \|_\alpha < 1$：最大 0.56 で条件満足 ✓

この厳密な検証の上で、**測地線方程式(フィンスラー版、チャーン接続を使用)を数値的に解き、31 ウェイポイントに離散化して FMS(飛行管理システム)に入力**する形で、実際の飛行経路が生成されました。

### *14 体の AI Agent*

ミッション中、14 体の AI Agent が並列に稼働します:

- **Embodied(機上)3 体**：Agent_Aero_Body(大気・乱気流検出)、Agent_Aero_Nav(測位統合)、Agent_Aero_Comms(通信管理)
- **Disembodied(地上)11 体**：Agent_Wx(気象)、Agent_ETOPS(代替空港)、Agent_Insurance(保険ファイバー)、Agent_Geopolitics(地政学)、他

特に、**ETOPS 穴**(緊急着陸可能空港までの距離制約)と**保険穴**(AP 条項区域への接近)という **2 つの位相的な穴**を**ジグザグ・パーシステンス**(Carlsson & de Silva, 2010)で同時追跡。2 つの穴が同時にゼロにならない時間帯の存在を、数学的に検出しました。

### *形式検証*

すべての計算の最後に、**Lean 4**(定理証明支援系)が走り、以下の命題を形式検証します:

- Randers condition satisfied（ランダー条件満足）
- Geodesic equation solved（測地線方程式が解けている）
- ETOPS corridor maintained（ETOPS 通廊維持）
- 248 boundary conditions handled（248 個の境界条件処理）
- Fallback triggered correctly（フォールバック発動の正しさ）
- All human final decisions recorded（最終判断は人間）

これらが通過した後に、**ZKML(ゼロ知識証明による機械学習の計算検証)の証明書**が生成され、CAA(英国民間航空局)への信頼性レポートに添付されます。

### *「測地線を信じますか」*

ケーススタディ最後の章で、ベテラン機長の白石遥は研修生にこう問います:

> 「測地線を信じますか」

これが、このケーススタディの核心の問いです。**数学的に最適な経路が証明されても、最終判断は人間が下す**。AI は測地線を計算し、人間が決断する。

GI 理論は「AI が経営判断を自動化する」ためのものではなく、「**人間の判断力をより深く支える数学的な土台**」なのです。この姿勢が、第 1 章で説明した共変微分・Lie 微分・測地線・最適制御という道具の、真の使い方です。

---

### 3.6 物理・工学での「当たり前」をまとめる

| 分野 | 多様体 | 計算されているもの |
|---|---|---|
| 一般相対性理論 | 4 次元時空 | アインシュタイン方程式、測地線 |
| GPS 補正 | 地球周辺の時空 | 測地線、時間膨張 |
| 量子重力 | より高次元の時空 | 曲率テンソル、ループ量子重力 |
| 創薬・タンパク質 | コンフォメーション空間 | 測地線、自由エネルギー最小化 |
| 神経科学 | 脳内の神経繊維空間 | Sub-Riemannian 測地線 |
| ロボット制御 | $T^n$、$SE(3)$ | 測地線、最適制御、Lie 微分 |
| 航空(GI ケーススタディ 11) | フィンスラー多様体 | ランダー測地線、$H^1$、ジグザグ |
| 量子計算 | Riemannian 複素多様体 | Fubini-Study 計量、最適ゲート |

**これら全部が、第 1 章で説明した GI 理論の道具の直接の適用です。**

GI 理論の革新性は「新しい数学を発明した」ことではありません。**「100 年前からある数学を、社会データに持ち込んだ」**ことです。なぜそれが新しいのか。なぜこれまで誰もやらなかったのか。それが第 4 章のテーマです。

---

## 第 4 章　なぜ社会・政治・経済データでは、これが今までできなかったのか

物理学・化学・工学の世界では、100 年以上前から多様体の上の計算が日常的です。ところが、**社会・経済・政治・文化データの分析では、これがほぼ全く行われてこなかった。**

GI 理論の開発者は、この理由を「**4 層の構造的ギャップ**」と呼んでいます。順に見ていきます。

### 4.1 第 1 層：社会科学におけるデータ制約と教育ギャップ

**データ制約**: 物理学は毎秒何十万回の実験データが取れます。化学の反応速度は 1 秒以下で測れます。ところが、社会科学の主要データは:

- GDP 統計：**四半期ごと**(3 ヶ月に 1 度しか更新されない)
- 失業率：月次
- 世論調査：月次から不定期
- 選挙結果：4 年に 1 度

データの粒度が粗く、量が少ない。多様体を厳密に推定するには、**ノイズに対して十分な密度**が必要なのに、社会データではそれが得にくい。

**教育ギャップ**: 経済学部・経営学部・政治学部のカリキュラムで、**微分幾何学は教えられていません**。統計学、計量経済学、機械学習までは教えます。しかし、計量テンソル、共変微分、Lie 微分、測地線──これらを学ぶ機会が社会科学の大学院にはありません。

結果、「データがあっても、それを曲がった空間として扱う発想も技術も持ち合わせていない」という状況が続いてきました。

### 4.2 第 2 層：工学・物理学における「既知多様体仮定」

工学と物理学は多様体の数学を使いますが、**対象の多様体はすでに分かっている**ことがほとんどです。

- ロボット制御：構成空間 $SO(3)$、$SE(3)$ は**理論から決まっている**
- 一般相対性理論：時空 $M^4$ は**アインシュタイン方程式の解として与えられる**
- 分子動力学：コンフォメーション空間は**化学結合から決まる**

つまり、これらの分野での問題は「**既知の多様体の上で、どう計算するか**」です。

ところが、社会データの場合:

> **「多様体がまだ存在しない。データから推定しなければならない」**

これは、物理学や工学にはなかった問題設定です。「多様体を**データから学習する**」という技術を、誰も準備してきませんでした。

### 4.3 第 3 層：機械学習における「2 つの多様体の混同」

ここが最も重要な層です。

機械学習の業界では、**Natural Policy Gradient(NPG)、TRPO、PPO** といった強化学習のアルゴリズムで「多様体上の最適化」が 1990 年代末から盛んに研究されてきました。

しかし、**彼らが扱っている多様体は違う多様体です。**

機械学習で扱われる多様体: - **方策パラメータ多様体** $\mathcal{M}_{\text{weights}}$ - ニューラルネットワークの**重みパラメータ**がなす多様体 - Fisher 情報計量などを使ってこの空間で最適化する

GI 理論で扱う多様体: - **環境状態多様体** $\mathcal{M}_{\text{env}}$ - 経営環境・政策環境そのものがなす多様体 - pullback 計量でこの空間の幾何を推定する

**この 2 つは全く違う多様体です。**

「AI のパラメータ空間」と「現実世界の状態空間」を混同してしまうと、NPG でやっているのだから GI 理論は不要、と早合点してしまう。実際、機械学習の主要な研究者の多くがこの混同をしており、**「環境自体の多様体の幾何学を推定する」という問題設定に、学術コミュニティが気づくのが遅れました。**

### 4.4 第 4 層：学際的交差問題

GI 理論を実装するには、**以下の 4 分野すべてに同時に熟達する必要があります。**

| 分野 | 必要な知識 |
|---|---|
| 微分幾何学 | リーマン計量、クリストッフェル記号、曲率テンソル、Lie 微分、測地線、フィンスラー計量、$H^1$コホモロジー |
| VAE・Neural ODE | 変分推論、自動微分、ヤコビ行列計算、$C^3$級活性化関数の選択、SVD によるフルランク検証 |
| 深層強化学習 | AI Agent 設計、World Models、並列 AI Agent 通信プロトコル |
| 領域知識 | 製造業、金融、半導体、航空、防衛──それぞれの業界のデータの特性と法規制 |

2026 年現在、**世界のどの大学も、この 4 分野を統合して教えるカリキュラムを持っていません。**個別の分野は:

- 微分幾何学：数学科・物理学科
- VAE：コンピュータサイエンス
- 深層強化学習：機械学習・人工知能
- 領域知識：MBA、経済学部、医学部

このような縦割りの結果、「**どれか 1 つには詳しいが、4 つ全部を統合できる人がいなかった**」。これが、GI 理論のような統合フレームワークが登場するのが遅れた最も根本的な理由です。

### 4.5 なぜ今、それが可能になったか

2020 年代半ばになって、この 4 分野の統合が可能になる環境がようやく整いました:

1. **VAE の普及**(Kingma & Welling 2014)で、データから潜在空間を構成する技術が成熟
2. **自動微分**(PyTorch、JAX)の発達で、ヤコビ行列・クリストッフェル記号・曲率テンソルを手計算なしで計算可能
3. **Neural ODE**(Chen et al. 2018)で、多様体上の ODE と Pontryagin の最大原理が接続
4. **ZKML・形式検証**(Lean 4、Z3)で、計算の正しさを暗号学的に保証可能

これらが揃ったのが、ほんの数年前です。

GI 理論は、この数年のうちに、**社会データの上で「物理学をする」ための道具が揃った**ことに最速で気づき、体系化したフレームワークだと言えます。

### 4.6 GI 理論の本質的な価値

以上を踏まえると、GI 理論の本質は次の一行に集約されます:

> **「経営環境・政策環境は、直線的(ユークリッド的)ではなく、曲がっている(リーマン的ないしフィンスラー的)。この曲がりを無視した分析は、地球表面を平面地図で航行するのと同種の構造誤差をもたらす。」**

100 年前、物理学者は「時空は曲がっている」ことに気づき、一般相対性理論を作りました。

現在、GI 理論は「**社会・経済・政治環境も曲がっている**」ことに気づき、その曲がりをデータから計算する方法を提示しました。

この視点の転換が、以下の具体的な能力につながります:

- 「同じ 1%が、場所によって違う意味を持つ」ことが定量化できる(気づき 1)
- 「施策の組み合わせの順序」が最適化できる(気づき 2)
- 「組織の断絶」を位相的に検出できる(気づき 3)
- 「危機の前兆」をスカラー曲率で監視できる(気づき 4)

- 「サプライチェーンの脆弱性」を曲率で評価できる(気づき 9)
- 「業界の分裂」を $H_1$で検出できる(気づき 10)
- 「航空路が封鎖される世界」を $H^1$コホモロジーで扱える(航空ケース)

ひとつひとつは、従来の分析手法では見えにくかったものばかりです。

---

## 第 5 章　代替アプローチ—「より良い道」はあるのか

第 1 章〜第 4 章では、GI 理論が現在採用している **VAE + pullback 計量**というアプローチを詳しく説明しました。

ここで、正直な問いを立てます:

> **数学者から「その基盤は不完全だ」と指摘されないための、より良いアプローチは存在するのか?**

この問いに真摯に答えるのが、本章の目的です。結論から言えば、**VAE + pullback 計量は、適切な注意事項を明記すれば数学的に批判に耐えます**。同時に、**いくつかの代替アプローチが、異なる強みをもって存在します**。各アプローチを、数学的に厳密な言葉で、しかし高校生にもわかるよう丁寧に紹介します。

### 5.1 VAE + pullback 計量の数学的地位—何が正しくて、何に注意すべきか

まず、現在のアプローチの正当性を明確にしましょう。

VAE のデコーダ $f_\theta: Z \to X$ が滑らかであれば、pullback 計量

$$g_{ij}(z) = \left(\frac{\partial f_\theta}{\partial z^i}\right)^\top \left(\frac{\partial f_\theta}{\partial z^j}\right) = \left(J_{f_\theta}^\top J_{f_\theta}\right)_{ij}$$

は潜在空間 $Z$ の上に**正定値対称テンソル場**を定義します。**これはリーマン計量の定義を満たします**。したがって、クリストッフェル記号、共変微分、リーマン曲率テンソル、スカラー曲率、Lie 微分のすべてが、標準的な微分幾何学の手続きで計算可能です。

高校生向けの言い方をすると:

> 「デコーダという滑らかな写像があれば、その微分(ヤコビ行列 $J$ )を使って $J^\top J$ を計算することで、必ず正しい『距離の測り方(リーマン計量)』ができあがる」

これは、線形代数で習う**グラム行列**の性質から来ています。任意の行列 $J$ に対して、$J^\top J$ は対称かつ半正定値。さらに $J$ がフルランク(列が線形独立)なら、**正定値**になります。これはリーマン計量の定義そのものです。

**しかし**、以下の 4 つの注意事項があります。数学者はここを指摘してくるでしょう。

### *注意事項 1：滑らかさの保証*

VAE のデコーダがニューラルネットワークで実装される場合、**ReLU 活性化関数**は微分が不連続です。ReLU は:

$$\mathrm{ReLU}(x) = \max(0, x)$$

と定義され、$x = 0$ の点で傾きが 0 から 1 に**ジャンプ**します。これは高校で習う「微分可能」の条件を満たしていません。

pullback 計量の定義にはデコーダの偏微分が必要なので、ReLU を使うと、**デコーダの「折れ目」の場所で計量が定義されなくなる**。さらに、曲率テンソルの計算には **2 階微分**が必要ですが、ReLU は 2 階微分が至る所でゼロ(ほとんどの場所)か無限大(折れ目の場所)になります。

**対策**: $C^\infty$ 級の活性化関数を使う:

- **tanh**(双曲線正接): $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$、あらゆる回数微分可能
- **Softplus**: $\mathrm{softplus}(x) = \log(1 + e^x)$、ReLU の滑らかな近似
- **GELU**: $\mathrm{GELU}(x) = x \cdot \Phi(x)$、$\Phi$ は標準正規分布の累積分布関数
- **SiLU / Swish**: $\mathrm{SiLU}(x) = x \cdot \sigma(x)$、$\sigma$ はシグモイド関数

これらを**全層で使用**すれば、デコーダは $C^\infty$ 級となり、曲率計算に十分な滑らかさが得られます。

### *注意事項 2：pullback 計量の退化*

デコーダの**ヤコビアンのランクが落ちる点**では、pullback 計量 $g = J^\top J$ が退化します。

これは高校数学の言葉で言うと、「デコーダが、潜在空間のある方向の動きにまったく反応しなくなる」状態です。たとえば、潜在変数の第 3 次元を 0.1 動かしても、出力画像が全然変わらない。こういう場所では:

- $g_{ij}$ が**ゼロ固有値**を持つ
- 逆行列 $g^{ij}$ が**存在しない**

- クリストッフェル記号が**定義できない**

**対策**: 特異値分解(SVD)で常時監視する。ヤコビ行列 $J$ を SVD 分解:

$$J = U\Sigma V^\top$$

$\Sigma$ は対角行列で、対角成分が特異値 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_d \geq 0$。**最小特異値** $\sigma_{\min}(J) = \sigma_d$ **を監視**し、閾値(たとえば $10^{-4}$ )以下の領域を「信頼できない領域」としてフラグを立てる。

GI ケーススタディ『偏西風の逆関数』では、最小特異値 $\sigma_{\min} = 0.312$ が確認されており、十分に安全な値です。

### *注意事項 3：潜在空間の位相*

VAE の潜在空間は、通常 $\mathbb{R}^d$ (ユークリッド空間)と**仮定**されます。しかし、データの本質的な構造が $\mathbb{R}^d$ ではない場合があります。

例: - **周期的な現象**(季節性のある経済データ、日周期のトラフィック)→ トーラス $T^d = (S^1)^d$ が自然 - **角度データ**(風向、方位)→ 球面 $S^2$ が自然 - **階層構造のあるデータ**(企業の組織図、系統樹)→ 双曲空間 $\mathbb{H}^d$ が自然

こうした場合、$\mathbb{R}^d$ を潜在空間にすると、**位相的なミスマッチ**が生じます。たとえば、周期的なデータで $\theta = 0$ と $\theta = 2\pi$ は「同じ点」ですが、$\mathbb{R}^d$ ではこの 2 点は無限に遠い。

**対策**: データの性質に応じて潜在空間の位相を選ぶ。**周期 VAE**(torus-VAE)、**球面 VAE**(sphere-VAE)、**双曲 VAE**(Poincaré embedding)など、近年さまざまな変種が提案されています。

### *注意事項 4：推定された計量と真の計量の関係*

これが最も重要な認識論的注意事項です。

pullback 計量 $g_{ij}(z) = (J^\top J)_{ij}$ は、**有限個のデータから VAE を訓練して得られた、デコーダ** $f_\theta$ **に依存する計量**です。真の社会経済現象が持つ(もし存在するなら)「真の計量」の**推定値**に過ぎません。

数学者は「これは真の計量か?」と問うかもしれません。正直な答えは:

> **「真の計量が存在するかどうかは、経験的問題です。私たちが扱っているのは、データから学習された近似的な計量であり、データが増えれば・モデルが良くなれば、真の計量に収束する(といいな、という信念を持っている)。」**

このことを**論文に明示**しておけば、数学者からの「貴方の計量は真の計量ではない」という指摘に対しては、「はい、そうです。これは推定値です」と答えられます。これは科学的誠実性の問題です。

---

### 5.2 代替アプローチ 1：情報幾何学(Information Geometry)

次に、**VAE + pullback 計量の代替**として考えられるアプローチを、5 つ紹介します。

最初は、甘利俊一(Amari)によって体系化された**情報幾何学**です。

#### *基本アイデア*

データを「点の集まり」として扱うのではなく、「**確率分布の族**」として扱います。

たとえば、ある国の経済状態を 1 次元の数値で表すのではなく、「その国の GDP、失業率、インフレ率の**同時確率分布** $p(x|\theta)$」として表現する。$\theta$ はこの分布を特徴付けるパラメータ。経済状態の変化 = パラメータ $\theta$ の変化。

すると、**パラメータ空間 $\{\theta\}$ そのものが多様体**になる。そして、この多様体の上には、**自然な計量**が存在します。

#### *Fisher 情報計量*

情報幾何学の核心は、**Fisher 情報計量**です。

$$g_{ij}(\theta) = E\left[\frac{\partial \log p(x|\theta)}{\partial \theta^i} \cdot \frac{\partial \log p(x|\theta)}{\partial \theta^j}\right]$$

高校生向けの解釈:

> 「パラメータ $\theta$ を少し変えたとき、確率分布 $p$ がどれくらい変わるか」の期待値。変化が大きい方向ほど、計量が大きい = 「敏感な方向」。

この計量は、以下の際立った数学的性質を持ちます:

1. **常に正定値**(確率分布の族が重なり合っていない限り)
2. **パラメータの取り方に依存しない**(座標変換で共変的に変わる)
3. **Cramér-Rao 不等式**という統計的不等式と直接結びつく

### *VAE + pullback 計量との比較*

| 観点 | VAE + pullback 計量 | 情報幾何学 |
|---|---|---|
| **数学的厳密性** | デコーダの滑らかさとフルランクが必要 | Fisher 計量は自動的にリーマン計量 |
| **滑らかさ** | 活性化関数の選択に依存 | 確率分布族の滑らかさから自動的に従う |
| **社会データへの適合** | 任意のデータに柔軟に適用可能 | データが確率分布として適切にモデル化できる必要がある |
| **計算コスト** | 中(VAE 訓練) | 低〜中(解析的に計算可能な場合が多い) |

### *高校生向けのたとえ*

VAE + pullback 計量が「**絵の具のパレットを描き起こしてから距離を測る**」のに対し、情報幾何学は「**そもそもデータを確率的な影として見て、影の形の空間の距離を直接測る**」アプローチです。

どちらも数学的には正当ですが、**アプローチの発想点が違う**のです。

### *興味深い接続*

実は、VAE の pullback 計量と Fisher 情報計量は、**特定の条件下で一致する**ことが知られています。VAE のエンコーダ $q(z|x)$ が定義する確率分布の Fisher 計量と、デコーダの pullback 計量は、**双対的な関係**にあります。

したがって、「どちらが正しいか」ではなく、「**同じ幾何学的対象を、違う視点から見ている**」のです。

---

## 5.3 代替アプローチ 2：最適輸送(Optimal Transport)

二つ目は、セドリック・ヴィラニ(Villani)らが深化させた**最適輸送理論**です。

### *基本アイデア*

「2 つの確率分布を比較する」方法として、**一方を他方に変形するのに必要な最小コスト**を計算する。

高校生向けの直観:

砂山 $A$ と砂山 $B$ がある。$A$ の砂を移動させて $B$ の形にしたい。砂を運ぶコスト(距離 × 重さ)を最小にする運び方が「最適輸送」。この最小コストが、$A$ と $B$ の「形の違い」を測る距離 = **Wasserstein 距離**。

### *Wasserstein 計量と Otto 微分*

Wasserstein 距離は以下で定義されます:

$$W_p(\mu,\nu) = \left( \inf_{\pi \in \Pi(\mu,\nu)} \int |x-y|^p d\pi(x,y) \right)^{1/p}$$

ここで $\Pi(\mu,\nu)$ は、周辺分布が $\mu$ と $\nu$ である結合分布全体。

**驚くべきこと**: Felix Otto(2001)が示したように、**確率分布全体の空間(Wasserstein 空間)は、形式的にリーマン多様体の構造を持ちます**。しかも**無限次元**の!

これは「Otto calculus」として知られ、以下が可能になります:

- Wasserstein 空間上の**測地線**(最も経済的に分布を変形する経路)
- Wasserstein 空間上の**勾配流**(熱方程式、Fokker-Planck 方程式は Wasserstein 勾配流)
- 輸送コスト最小化の**双対問題**(Kantorovich-Rubinstein 定理)

### *VAE + pullback 計量との比較*

| 観点 | VAE + pullback 計量 | 最適輸送 |
|---|---|---|
| **数学的厳密性** | 高い(注意事項あり) | 非常に高い。数学的に厳密 |
| **潜在空間の仮定** | $\mathbb{R}^d$ を仮定 | 不要 |
| **時系列への適用** | 自然 | 分布の時間変化として扱える |
| **計算コスト** | 中 | 高(高次元データでは特に) |
| **Lie 微分・スカラー曲率** | 自然に計算可能 | Wasserstein 空間上での実用的計算は研究の前線 |

### *独自の強み*

最適輸送は「**分布 $A$ から分布 $B$ への変化の量**」を、極めて厳密に測定します。このため、以下に特に向いています:

- **二時点比較**(2019 年の経済 vs 2024 年の経済)
- **生成モデルの品質評価**(WGAN の核心)

- **異常検知**(異常データ点 = 通常分布から遠いデータ点)

*高校生向けのたとえ*

VAE + pullback 計量が「**曲がった地面の上で、A 地点から B 地点への道のり**」を測るのに対し、最適輸送は「**A 町の建物を全部 B 町に引っ越す最小コスト**」を測る。前者は**点と点**の距離、後者は**町と町**の距離を扱います。

---

### 5.4 代替アプローチ 3：拡散幾何学(Diffusion Geometry)

三つ目は、コイフマン(Coifman)とラフォン(Lafon)の**拡散幾何学**です。

*基本アイデア*

データ点の間に「熱が伝わるように情報が広がる」プロセスを定義し、**熱の広がり方からデータの構造を推定する**。

高校生向けの直観:

> 氷の上にインクを一滴落とす。インクは周囲に広がっていく。氷の形(平らか、凹凸があるか、穴があるか)によって広がり方が違う。この違いから氷の形を逆算する。

*拡散作用素と固有値分解*

データ点 $\{x_1, \dots, x_N\}$ に対し、ガウス核で類似度行列を構成:

$$W_{ij} = \exp\left(-\frac{\| x_i - x_j \|^2}{\epsilon}\right)$$

この $W$ から確率遷移行列(マルコフ連鎖の 1 ステップ推移)を作る:

$$P = D^{-1}W$$

ここで $D$ は度数行列(対角行列で $D_{ii} = \sum_j W_{ij}$ )。

$P$ を $t$ 回適用した $P^t$ の行列成分 $P_{ij}^t$ が、「点 $x_i$ から出発した熱が、時刻 $t$ に点 $x_j$ にいる確率」を表す。2 点間の**拡散距離**:

$$D_t(x_i, x_j)^2 = \sum_k \frac{\left(P_{ik}^t - P_{jk}^t\right)^2}{\phi_0(k)}$$

この拡散距離が、**データの内在幾何学を反映した距離**になる。

### *数学的長所*

- **ノイズに対して頑健**: 拡散プロセスがノイズを自然に平滑化
- **連続極限で Laplace-Beltrami 作用素に収束**: 多様体仮説が成立する場合、理論的保証がある(Belkin & Niyogi, 2008)
- **計算が比較的安価**: 固有値分解の計算コスト

### *VAE + pullback 計量との比較*

| 観点 | VAE + pullback 計量 | 拡散幾何学 |
|---|---|---|
| **ノイズへのロバスト性** | VAE 訓練に依存 | 高い(拡散がノイズを平滑化) |
| **スケーラビリティ** | 高い(GPU 並列) | 中(固有値分解コスト) |
| **動的データ** | 時系列で自然 | 時系列への拡張は発展途上 |
| **解釈可能性** | 中 | 高い(スペクトル解析) |

### *高校生向けのたとえ*

VAE + pullback 計量が「**データから CG を描き起こして、その CG の表面の距離を測る**」のに対し、拡散幾何学は「**データにインクを垂らして、どう広がるかでデータの形を推測する**」。後者は**直接データに働きかける**のが特徴です。

---

## 5.5 代替アプローチ 4：ニューラル ODE / 正規化フロー(Neural ODE / Normalizing Flows)

四つ目は、チェン(Chen)らが 2018 年に発表した**ニューラル ODE** です。これは、GI 理論の数学的基盤を最も強く強化する可能性を秘めています。

### *基本アイデア*

VAE のデコーダを「**一気に高次元空間に展開する**」のではなく、「**時間をかけて少しずつ変形していく**」アプローチ。

デコーダを常微分方程式(ODE)の解として表現:

$$\frac{dz(t)}{dt} = f_\theta(z(t), t), \quad z(0) = z_0$$

これを時刻 $t = T$ まで解くと、最終値 $z(T)$ がデータ空間の点になります。

### *微分同相性が構造的に保証される*

ここが数学的に決定的に重要です。

ODE の右辺 $f_\theta$ が **Lipschitz 連続**(穏やかな条件)なら、解写像 $z_0 \mapsto z(T)$ は**微分同相写像(diffeomorphism)**であることが保証されます。これは:

- **滑らか**(ODE の右辺の滑らかさから従う)
- **単射**(同じ ODE の異なる初期値は異なる終端値)
- **フルランク**(退化しない)

つまり、**GI 命題 1.1 の条件(i)〜(iv)が構造的に自動満足される**のです!

これは、VAE + pullback 計量が「4 条件を手で検証する」必要があるのに対し、ニューラル ODE は「4 条件を忘れても大丈夫」という強い数学的保証を与えます。

### *随伴方程式 = ポントリャーギン随伴方程式*

さらに、ニューラル ODE の勾配計算には**随伴方程式**:

$$\frac{da(t)}{dt} = -a(t)^\top \frac{\partial f_\theta}{\partial z}$$

が使われます。$a(t) = \partial L / \partial z(t)$ は随伴状態。

**驚くべきこと**: この随伴方程式は、**ポントリャーギンの最大原理の随伴方程式と数学的に等価**です。つまり、ニューラル ODE で訓練した時点で、**最適制御の枠組みが自動的に組み込まれている**のです。

GI ケーススタディ『偏西風の逆関数』の「12 時間の随伴変数」という副題は、まさにこの随伴変数を指しています。

### *VAE + pullback 計量との比較*

| 観点 | VAE + pullback 計量 | ニューラル ODE |
|---|---|---|
| **数学的保証** | 4 条件の手動検証 | 微分同相性が構造的に自動保証 |
| **滑らかさ** | 活性化関数依存 | ODE 右辺の滑らかさから自動 |
| **制御との接続** | 間接的 | 直接的(随伴 = ポントリャーギン) |
| **計算コスト** | 中 | 高(ODE 数値解法) |
| **次元削減** | 明示的(低次元潜在空間) | ODE は次元を保存するため、別途の工夫が必要 |

### *独自の強み*

- **数学的保証の強さ**: 微分同相性、滑らかさ、単射性すべてが自動
- **最適制御との自然な接続**: 随伴方程式が標準的

- **長期シミュレーションでの安定性**: シンプレクティック積分法と組み合わせると、さらに強固

*高校生向けのたとえ*

VAE が「**粘土をぐにゃっと一発で目的の形にする**」のに対し、ニューラル ODE は「**粘土を少しずつ、連続的に変形していく**」。途中で粘土がちぎれたり、自己交差したりしない保証があるのが、後者の強みです。

---

### 5.6 代替アプローチ 5：ガウス過程潜在変数モデル(GPLVM)

五つ目は、ローレンス(Lawrence, 2005)が提案した **GPLVM(Gaussian Process Latent Variable Model)**です。

*基本アイデア*

潜在空間からデータ空間への写像を、**ガウス過程**で定義する。ガウス過程は、「関数に対する確率分布」という概念で、以下の性質を持ちます:

- 任意の有限点集合での値が**多変量正規分布**に従う
- **カーネル関数**で共分散構造が決まる
- **不確実性が自然に定量化**される(各予測に信頼区間が付く)

*数学的長所*

- **滑らかさが保証される**: カーネル関数を滑らかに選べば(RBF カーネル等)、写像が自動的に $C^\infty$
- **不確実性の定量化**: 各点で「どれくらい確信があるか」が分散として出てくる
- **pullback 計量が解析的に計算可能**: カーネル関数から直接導出できる場合がある

*VAE + pullback 計量との比較*

| 観点 | VAE + pullback 計量 | GPLVM |
|---|---|---|
| **不確実性の定量化** | MC Dropout 等の追加的工夫が必要 | 組み込まれている |
| **滑らかさ** | 活性化関数依存 | カーネル選択から自動的 |
| **スケーラビリティ** | 高 | 低(データ数の 3 乗の計算量) |
| **解析的性質** | 数値計算中心 | 解析的計算が多く可能 |

*高校生向けのたとえ*

VAE が「**1 本の鉛筆で素早くスケッチ**」するのに対し、GPLVM は「**水彩絵の具で、にじみ(不確実性)も含めて丁寧に描く**」。後者は時間がかかるが、「自信がある場所」と「自信がない場所」が絵の中に反映される。

---

### 5.7 5 つのアプローチの総合比較

| 判断基準 | 最適なアプローチ |
|---|---|
| 数学的厳密性を最優先 | ニューラル ODE |
| 大規模データへのスケーラビリティ | VAE + pullback 計量 |
| 統計的推論との接続 | 情報幾何学 |
| ノイズへのロバスト性 | 拡散幾何学 |
| 不確実性の定量化 | GPLVM |
| 分布間の距離の厳密な測定 | 最適輸送 |
| 制御・政策介入シミュレーション | ニューラル ODE |
| 実装の容易さ | VAE + pullback 計量 |
| 長期時系列の連続追跡 | ニューラル ODE |

### 5.8 では、結局どれを使うべきか

正直な結論は:

1. **現在の VAE + pullback 計量は、4 つの注意事項を明記すれば数学的に批判に耐えます。**
2. **数学的基盤をさらに強化したいなら、ニューラル ODE が最も有望**です。微分同相性の自動保証と、ポントリャーギンの最大原理との直接的な接続は、GI 理論の野心と完璧に整合します。
3. **複数アプローチの併用が、実務的には最強**です。VAE で最初の多様体を構成し、拡散幾何学で独立検証し、ニューラル ODE で制御シミュレーションを行う──こうした**スタック型アプローチ**が、2026 年現在、GI 理論の実装として最も成熟した形になります。

この姿勢を、GI ケーススタディ『偏西風の逆関数』は **Vector Diffusion Maps による独立検証**として既に先取りしています。Vector Diffusion Maps(Singer & Wu, 2012)は拡散幾何学の一種であり、VAE で構成した計量の**独立な検証**として機能しているのです。

これが、「数学者から誤りを指摘されない」ための最善策です。単独のアプローチに依存するのではなく、**複数のアプローチで互いに検証する**──これが、科学的に誠実な道です。

---

# 第 6 章　Python 実装例—実際にコードで書いてみる

第 1 章〜第 5 章で説明した概念を、Python で実装する具体例を示します。これは「どのモジュールを使うか」「どのような構造のコードになるか」を実感してもらうための教育的な例です。

## 6.1 使用モジュールの一覧

| モジュール | 用途 | インストール |
|---|---|---|
| `torch` (PyTorch) | VAE の実装、自動微分、ヤコビ行列計算 | `pip install torch` |
| `torchdiffeq` | ニューラル ODE の実装 | `pip install torchdiffeq` |
| `gpytorch` | GPLVM の実装 | `pip install gpytorch` |
| `POT` (Python Optimal Transport) | 最適輸送(Wasserstein 距離) | `pip install POT` |
| `scikit-learn` | 前処理、PCA、拡散写像の一部 | `pip install scikit-learn` |
| `pydiffmap` | 拡散写像(Diffusion Maps) | `pip install pydiffmap` |
| `numpy`, `scipy` | 数値計算の基礎 | `pip install numpy scipy` |
| `matplotlib`, `plotly` | 可視化 | `pip install matplotlib plotly` |
| `shap` | 説明可能性(SHAP) | `pip install shap` |
| `econml` | 因果推論(DML) | `pip install econml` |

## 6.2 VAE の実装(滑らかな活性化関数を使う)

GI 理論の命題 1.1 の条件(i)を満たすため、**tanh** を使用します。ReLU は使用しません。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SmoothVAE(nn.Module):
```

```
    """GI 理論のための VAE。tanh を使って C^∞ 級を保証する。"""

    def __init__(self, input_dim, latent_dim, hidden_dim=128):
        super().__init__()
        self.latent_dim = latent_dim

        # エンコーダ: x -> (mu, log_var)
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Tanh(),  # C^∞ 級の活性化関数
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
        )
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_log_var = nn.Linear(hidden_dim, latent_dim)

        # デコーダ: z -> x_reconstructed
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, input_dim),
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_log_var(h)

    def decode(self, z):
        return self.decoder(z)

    def reparameterize(self, mu, log_var):
        std = torch.exp(0.5 * log_var)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        mu, log_var = self.encode(x)
        z = self.reparameterize(mu, log_var)
        x_reconstructed = self.decode(z)
        return x_reconstructed, mu, log_var

def vae_loss(x_reconstructed, x, mu, log_var):
    """VAE の損失関数: 再構成誤差 + KL ダイバージェンス"""
```

```
    recon_loss = F.mse_loss(x_reconstructed, x, reduction='sum')
    kl_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return recon_loss + kl_loss
```

### 6.3 Pullback 計量の計算

訓練済み VAE のデコーダから pullback 計量 $g_{ij}(z) = (J^\top J)_{ij}$ を計算します。**torch.autograd.functional.jacobian** を使って、自動微分でヤコビ行列を求めます。

```
def compute_pullback_metric(decoder, z):
    """
    潜在空間の点 z における pullback 計量 g_{ij} = (J^T J)_{ij} を計算。

    Args:
        decoder: 訓練済み VAE のデコーダ (nn.Module)
        z: 潜在空間の点 (torch.Tensor, shape=(latent_dim,))

    Returns:
        g: pullback 計量 (torch.Tensor, shape=(latent_dim, latent_dim))
    """
    z = z.clone().requires_grad_(True)

    # ヤコビ行列 J = ∂f/∂z (shape: (data_dim, latent_dim))
    J = torch.autograd.functional.jacobian(decoder, z)

    # pullback 計量 g = J^T J
    g = J.T @ J

    return g

def check_metric_conditions(decoder, z_samples, threshold=1e-4):
    """
    命題1.1 の条件(iii)「フルランク」を検証する。
    全サンプル点でヤコビ行列の最小特異値 > threshold を確認。
    """
    min_sigmas = []
    for z in z_samples:
        J = torch.autograd.functional.jacobian(decoder, z)
        U, S, Vh = torch.linalg.svd(J, full_matrices=False)
        min_sigmas.append(S.min().item())

    min_sigma_global = min(min_sigmas)
    passed = min_sigma_global > threshold
```

```
    print(f"最小特異値(全サンプル): {min_sigma_global:.6f}")
    print(f"閾値: {threshold}")
    print(f"条件(iii) フルランク: {'✓ 満足' if passed else '✗ 違反'}")
    return passed, min_sigmas
```

### 6.4 クリストッフェル記号と共変微分の計算

Pullback 計量から、**クリストッフェル記号**を自動微分で計算します。これには `create_graph=True` が不可欠です(1 階微分の結果を再度微分する必要があるため)。

```
def compute_christoffel_symbols(decoder, z):
    """
    クリストッフェル記号 Γ^k_{ij} を計算。

    Γ^k_{ij} = (1/2) g^{kl} (∂g_{jl}/∂z^i + ∂g_{il}/∂z^j - ∂g_{ij}/∂z^l)
    """
    d = z.shape[0]  # 潜在空間の次元
    z = z.clone().requires_grad_(True)

    # pullback 計量 g_{ij}
    g = compute_pullback_metric_grad(decoder, z)  # create_graph=True で
計算

    # 逆計量 g^{ij}
    g_inv = torch.linalg.inv(g)

    # 計量の 1 階偏微分 ∂g_{ij}/∂z^k
    dg = torch.zeros(d, d, d)
    for i in range(d):
        for j in range(d):
            grads = torch.autograd.grad(
                g[i, j], z, create_graph=True, retain_graph=True
            )[0]
            dg[i, j, :] = grads

    # クリストッフェル記号を組み立てる
    Gamma = torch.zeros(d, d, d)
    for k in range(d):
        for i in range(d):
            for j in range(d):
                total = 0.0
                for l in range(d):
                    total += g_inv[k, l] * (dg[j, l, i] + dg[i, l, j] - d
g[i, j, l])
```

```
                Gamma[k, i, j] = 0.5 * total

    return Gamma

def compute_pullback_metric_grad(decoder, z):
    """create_graph=True 付きで pullback 計量を計算(高階微分のため)"""
    J = torch.autograd.functional.jacobian(decoder, z, create_graph=True)
    return J.T @ J
```

### 6.5 Lie 微分の計算

ベクトル場 $V$ に対する計量の Lie 微分 $(\mathcal{L}_V g)_{ij} = \nabla_i V_j + \nabla_j V_i$ を計算します。

```
def compute_lie_derivative(decoder, z, V_field):
    """
    Lie 微分 (L_V g)_{ij} = ∇_i V_j + ∇_j V_i を計算。

    Args:
        decoder: 訓練済み VAE のデコーダ
        z: 潜在空間の点
        V_field: ベクトル場を表す関数 V: R^d -> R^d

    Returns:
        L_V_g: Lie 微分 (torch.Tensor, shape=(latent_dim, latent_dim))
    """
    d = z.shape[0]
    z = z.clone().requires_grad_(True)

    # 計量とクリストッフェル記号
    g = compute_pullback_metric_grad(decoder, z)
    Gamma = compute_christoffel_symbols(decoder, z)

    # ベクトル場の値
    V = V_field(z)  # shape=(d,)

    # V の 1 階偏微分
    dV = torch.zeros(d, d)
    for i in range(d):
        grads = torch.autograd.grad(V[i], z, create_graph=True, retain_gr
aph=True)[0]
        dV[i, :] = grads

    # 共変微分 ∇_j V_i = ∂_j V_i - Γ^k_{ji} V_k
    nabla_V = torch.zeros(d, d)  # nabla_V[j][i] = ∇_j V_i
    for j in range(d):
```

```
        for i in range(d):
            correction = sum(Gamma[k, j, i] * V[k] for k in range(d))
            nabla_V[j, i] = dV[i, j] - correction

    # 共変微分を降下指数 V_i = g_{ij} V^j に変換
    V_down = g @ V  # V_i

    # Lie 微分 (L_V g)_{ij} = ∇_i V_j + ∇_j V_i
    L_V_g = nabla_V + nabla_V.T

    return L_V_g
```

### 6.6 測地線の数値計算

クリストッフェル記号を使って、**測地線方程式**を数値的に解きます。SciPy の solve_ivp を使用します。

```
from scipy.integrate import solve_ivp
import numpy as np

def geodesic_equation(t, state, decoder, latent_dim):
    """
    測地線方程式: d²γ^k/dt² + Γ^k_{ij} (dγ^i/dt)(dγ^j/dt) = 0

    state = [γ, γ_dot] の 2d 次元ベクトル
    """
    gamma = state[:latent_dim]
    gamma_dot = state[latent_dim:]

    # クリストッフェル記号
    z = torch.tensor(gamma, dtype=torch.float32)
    Gamma = compute_christoffel_symbols(decoder, z).detach().numpy()

    # 2 階微分 γ_ddot^k = -Γ^k_{ij} γ_dot^i γ_dot^j
    gamma_ddot = np.zeros(latent_dim)
    for k in range(latent_dim):
        for i in range(latent_dim):
            for j in range(latent_dim):
                gamma_ddot[k] -= Gamma[k, i, j] * gamma_dot[i] * gamma_do
t[j]

    return np.concatenate([gamma_dot, gamma_ddot])

def compute_geodesic(decoder, z_start, v_start, t_end=1.0, n_points=100):
    """
```

```
    測地線を数値計算する。

    Args:
        decoder: 訓練済みVAEのデコーダ
        z_start: 出発点 (numpy array)
        v_start: 出発速度 (numpy array)
        t_end: 終端時刻
        n_points: サンプル点数

    Returns:
        t_array, gamma_array: 時刻配列と測地線の点列
    """
    latent_dim = len(z_start)
    initial_state = np.concatenate([z_start, v_start])

    solution = solve_ivp(
        geodesic_equation,
        (0, t_end),
        initial_state,
        args=(decoder, latent_dim),
        t_eval=np.linspace(0, t_end, n_points),
        method='RK45',  # Runge-Kutta 4次
        rtol=1e-6
    )

    return solution.t, solution.y[:latent_dim].T
```

### 6.7 ニューラルODEによる代替実装

torchdiffeqを使うと、VAEを**ニューラルODE**で置き換えた実装が可能です。これは第5.5節で説明した、数学的に最強のアプローチです。

```
from torchdiffeq import odeint

class ODEFunc(nn.Module):
    """ニューラルODEの右辺 f_θ(z, t)"""

    def __init__(self, latent_dim, hidden_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, latent_dim),
```

```
        )

    def forward(self, t, z):
        return self.net(z)

class NeuralODEDecoder(nn.Module):
    """ニューラルODE によるデコーダ。微分同相性が構造的に保証される。"""

    def __init__(self, latent_dim, hidden_dim=128):
        super().__init__()
        self.ode_func = ODEFunc(latent_dim, hidden_dim)

    def forward(self, z0, t_end=1.0):
        t = torch.tensor([0.0, t_end])
        z_trajectory = odeint(self.ode_func, z0, t, method='dopri5')
        return z_trajectory[-1]  # 終端値
```

### 6.8 最適輸送(Wasserstein 距離) による独立検証

POT(Python Optimal Transport) を使って、2 つの時点のデータ分布間の Wasserstein 距離を計算します。

```
import ot
import numpy as np

def compute_wasserstein_distance(X_t1, X_t2):
    """
    時点 t1 のデータ X_t1 と時点 t2 のデータ X_t2 の間の
    Wasserstein-2 距離を計算。

    Args:
        X_t1, X_t2: データ点の集合 (numpy array, shape=(n, d))

    Returns:
        W2: Wasserstein-2 距離
    """
    n1, n2 = len(X_t1), len(X_t2)

    # 一様な重み
    a = np.ones(n1) / n1
    b = np.ones(n2) / n2

    # コスト行列(ユークリッド距離の 2 乗)
    M = ot.dist(X_t1, X_t2, metric='sqeuclidean')
```

```
    # EMD(Earth Mover's Distance)
    W2_squared = ot.emd2(a, b, M)

    return np.sqrt(W2_squared)
```

### 6.9 拡散写像による独立検証

pydiffmap を使って、VAE の pullback 計量を**拡散幾何学**で独立に検証します。

```
from pydiffmap import diffusion_map

def validate_with_diffusion_maps(X, n_components=3, epsilon='bgh'):
    """
    拡散写像でデータの低次元表現を求め、VAE の結果と比較する。

    Args:
        X: データ (numpy array, shape=(n, d))
        n_components: 埋め込み次元
        epsilon: バンド幅(自動選択: 'bgh')

    Returns:
        X_embedded: 低次元埋め込み
        dmap: 拡散写像オブジェクト
    """
    dmap = diffusion_map.DiffusionMap.from_sklearn(
        n_evecs=n_components,
        epsilon=epsilon,
        alpha=1.0,  # 正規化: 1.0 で Laplace-Beltrami に収束
        k=64  # 近傍数
    )
    X_embedded = dmap.fit_transform(X)
    return X_embedded, dmap
```

### 6.10 完全なパイプラインの例

以上の要素を組み合わせた、GI 理論の完全なパイプラインの骨格:

```
def gi_theory_pipeline(data, latent_dim=3, n_epochs=1000, lr=1e-3):
    """
    GI 理論の10ステップ・パイプラインの実装骨格。

    Args:
        data: 社会経済データ (numpy array, shape=(n, d))
        latent_dim: 潜在空間の次元

    Returns:
```

```
        results: 幾何学的量の辞書
    """
    input_dim = data.shape[1]
    X = torch.tensor(data, dtype=torch.float32)

    # Step 1-2: VAE の訓練
    vae = SmoothVAE(input_dim, latent_dim)
    optimizer = torch.optim.Adam(vae.parameters(), lr=lr)

    for epoch in range(n_epochs):
        optimizer.zero_grad()
        x_recon, mu, log_var = vae(X)
        loss = vae_loss(x_recon, X, mu, log_var)
        loss.backward()
        optimizer.step()

    # Step 3: pullback 計量の計算(サンプル点で)
    decoder = vae.decode
    z_samples = torch.randn(100, latent_dim)
    metrics = [compute_pullback_metric(decoder, z) for z in z_samples]

    # Step 4: フルランク条件の検証
    passed, min_sigmas = check_metric_conditions(decoder, z_samples)

    # Step 5-6: クリストッフェル記号とスカラー曲率の計算
    # (実装省略、上記関数を使用)

    # Step 7: Lie 微分(施策ベクトル場を定義して)
    # V_policy = lambda z: torch.tensor([0.1, 0.0, 0.0])  # 例: 第1次元方
向の施策
    # lie_derivatives = [compute_lie_derivative(decoder, z, V_policy) for
 z in z_samples]

    # Step 8: 測地線の計算
    z_start = np.zeros(latent_dim)
    v_start = np.array([1.0] + [0.0]*(latent_dim-1))
    t_array, gamma_array = compute_geodesic(decoder, z_start, v_start)

    # Step 9: 最適制御(ポントリャーギン最大原理)──ニューラルODE 推奨

    # Step 10: 独立検証(拡散写像)
    X_dmap, dmap = validate_with_diffusion_maps(data)

    return {
```

```
        'vae': vae,
        'pullback_metrics': metrics,
        'rank_check': passed,
        'min_singular_values': min_sigmas,
        'geodesic': (t_array, gamma_array),
        'diffusion_embedding': X_dmap,
    }
```

### 6.11 説明可能性(SHAP)と因果推論(DML)の統合

GI 理論の「5 本の柱」の実装例:

```
import shap
from econml.dml import LinearDML

def explain_with_shap(vae_model, X_background, X_explain):
    """SHAP で VAE の判断を要因分解する"""
    explainer = shap.DeepExplainer(vae_model, X_background)
    shap_values = explainer.shap_values(X_explain)
    return shap_values

def causal_analysis_with_dml(X, Y, T):
    """
    DML で因果効果を推定する。

    Args:
        X: 共変量
        Y: 結果変数
        T: 施策(treatment)
    """
    est = LinearDML(random_state=42)
    est.fit(Y, T, X=X)

    # 平均処置効果(ATE)
    ate = est.ate(X)

    # 条件付き平均処置効果(CATE)
    cate = est.effect(X)

    return ate, cate
```

### 6.12 実装上の注意点

最後に、実装で躓きやすいポイントを列挙します。

1. **`create_graph=True` を忘れない**: 2 階微分(曲率計算)には必須

2. **`.requires_grad_(True)`**: 自動微分したい変数に明示的に要求
3. **ReLU を絶対に使わない**: 代わりに tanh、Softplus、GELU 等の $C^\infty$ 級活性化関数
4. **条件数の監視**: `torch.linalg.cond(g)` で計量の条件数を監視し、$10^4$ を超えたら警告
5. **数値安定化**: 計量の逆行列計算は `torch.linalg.solve` を使う(inv は避ける)
6. **メモリ管理**: 2 階微分はメモリ消費が大きいため、`retain_graph=True` を計画的に使う

これらを守れば、数学者から「コードが数学を正しく実装していない」という指摘を受けることはないでしょう。

---

## 終わりに—「測地線を信じますか」

GI 理論のケーススタディ『偏西風の逆関数』の最後、機長の白石遥は研修生にこう問います:

> 「測地線を信じますか」

これは、GI 理論の本質を一言で表す問いです。

数学は正しい。しかし数学だけでは意思決定はできない。最終的に、目の前の状況で「この測地線を信じてこの経路を飛ぶか」を決めるのは人間です。

GI 理論は、「**人間の判断を置き換える AI**」を目指しているのではありません。「**人間の判断力をより深く支える、数学的に厳密な土台**」を提供するものです。

そのために、VAE で潜在空間を構成し、pullback 計量で距離を定義し、共変微分で本当の変化率を測り、Lie 微分で施策の構造変形力を定量化し、測地線で最適経路を描き、最適制御で最小コストの施策を計算する—

これらの道具は、100 年以上前から物理学者・工学者が使ってきたもの。GI 理論はそれを、「**社会・経済・政治・文化データ**」という、これまで曲がった空間として扱われてこなかった領域に、初めて体系的に持ち込むフレームワークです。

これが、VAE + pullback 計量という手間をかける意味です。この手間が、経営判断と政策決定を、「平面地図の誤差」から解放します。

---

# 参考文献

## 数学的基盤


- Arvanitidis, G., Hansen, L. K., & Hauberg, S. (2018). “Latent Space Oddity: On the Curvature of Deep Generative Models.” *ICLR.*
- Bao, D., Chern, S.-S., & Shen, Z. (2000). *An Introduction to Riemann-Finsler Geometry.* Springer.
- Do Carmo, M. P. (1992). *Riemannian Geometry.* Birkhäuser.
- Lee, J. M. (2012). *Introduction to Smooth Manifolds* (2nd ed.). Springer.
- Pontryagin, L. S., et al. (1962). *The Mathematical Theory of Optimal Processes.* Wiley.


## 代替アプローチ


- Amari, S. (2016). *Information Geometry and Its Applications.* Springer.
- Villani, C. (2009). *Optimal Transport: Old and New.* Springer.
- Coifman, R. R., & Lafon, S. (2006). “Diffusion maps.” *Applied and Computational Harmonic Analysis*, 21(1), 5-30.
- Belkin, M., & Niyogi, P. (2008). “Towards a theoretical foundation for Laplacian-based manifold methods.” *Journal of Computer and System Sciences*, 74(8), 1289-1308.
- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). “Neural Ordinary Differential Equations.” *NeurIPS.*
- Lawrence, N. D. (2005). “Probabilistic Non-linear Principal Component Analysis with Gaussian Process Latent Variable Models.” *JMLR*, 6, 1783-1816.
- Otto, F. (2001). “The geometry of dissipative evolution equations: the porous medium equation.” *Comm. Partial Differential Equations*, 26(1-2), 101-174.
- Singer, A., & Wu, H.-T. (2012). “Vector diffusion maps and the connection Laplacian.” *Comm. Pure Appl. Math.*, 65(8), 1067-1144.


## 関連手法


- Carlsson, G., & de Silva, V. (2010). “Zigzag persistence.” *Foundations of Computational Mathematics*, 10, 367-405.
- Chernozhukov, V., et al. (2018). “Double/debiased machine learning.” *The Econometrics Journal*, 21(1), C1-C68.
- Gal, Y., & Ghahramani, Z. (2016). “Dropout as a Bayesian approximation.” *ICML.*
- Kingma, D. P., & Welling, M. (2014). “Auto-encoding variational Bayes.” *ICLR.*
- Lundberg, S. M., & Lee, S.-I. (2017). “A unified approach to interpreting model predictions.” *NeurIPS.*

### GI 理論


- Étale Cohomology (2026a). *Geometric Intelligence, Volume 1.* Zenodo. https://doi.org/10.5281/zenodo.19140918
- Étale Cohomology (2026b). *Geometric Intelligence, Volume 2.* Zenodo. https://doi.org/10.5281/zenodo.19157891
- Étale Cohomology (2026c). *偏西風の逆関数——12 時間の随伴変数と 248 の境界条件* (GI ケーススタディ・ノベル第 11 作).


### Python 実装関連


- Paszke, A., et al. (2019). “PyTorch: An imperative style, high-performance deep learning library.” *NeurIPS.*
- Flamary, R., et al. (2021). “POT: Python Optimal Transport.” *JMLR*, 22(78), 1-8.
- Gardner, J., et al. (2018). “GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration.” *NeurIPS.*